

PROJECT DOCUMENTATION

Universal Web Scraper & Data Management System

Python • FastAPI • PostgreSQL • Playwright • React

Technical Approach, Architecture, Working Flow and API Documentation

Document Item	Details
Project Type	Full-stack web scraping and data management application
Backend	Python, FastAPI
Scraping	Requests + BeautifulSoup + Playwright
Database	PostgreSQL
Frontend	React + Vite + Tailwind CSS
Data Export	Python csv module + API download
Architecture	Layered architecture with Repository, Factory and Dependency Injection
Primary Objective	Scrape website content, persist it, expose CRUD/schema APIs and export CSV

Prepared for project submission and mentor/panel presentation.

1. Executive Summary

The Universal Web Scraper is a full-stack application developed to accept a website URL, retrieve website content, process the extracted information into a structured format, store the result in PostgreSQL, and expose APIs for data management and database schema inspection. The application supports both ordinary HTTP scraping and JavaScript-rendered pages through a browser automation fallback.

The system is designed as a reusable scraper rather than a scraper hard-coded for one website. The backend separates scraping, parsing, normalization, database access, business logic and API routing. A React frontend provides a simple interface for scraping websites, viewing records, editing/deleting data, inspecting the database schema and downloading CSV data.

2. Assignment Requirements Covered

The provided assignment requires dynamic/website scraping, PostgreSQL storage, CRUD APIs, schema APIs, a separate CSV generation script, GitHub submission, README documentation, a Loom demonstration, and documentation of the developed approach and flow.

Requirement	Implementation in Project
Accept a website URL	POST /scrape/ accepts a URL through a Pydantic request schema.
Scrape website data	Requests + BeautifulSoup; Playwright browser fallback for JavaScript-rendered pages.
Structured scraped data	ScrapedPage and ScrapedLink Pydantic models.
Store data in PostgreSQL	psycopg2 connection layer + scraped_pages table.
Create API	POST /data/
Read APIs	GET /data/ and GET /data/{record_id}
Update API	PUT /data/{record_id}
Delete API	DELETE /data/{record_id}
Schema APIs	GET /schema/tables and GET /schema/tables/{table_name}
CSV script	scripts/generate_csv.py using Python csv module.
CSV API	GET /export/csv provides downloadable CSV.
Documentation	README + this project document.

3. Problem Statement

Websites expose information in different forms. Some pages return useful HTML directly, while others depend on JavaScript to render content at runtime. The project addresses this problem by providing a common interface where a user submits a URL and the application attempts to collect useful page information in a structured format.

After extraction, the data is persisted in PostgreSQL so that it can be queried and managed independently of the original web page. CRUD and schema APIs make the stored information accessible to the frontend or any API client.

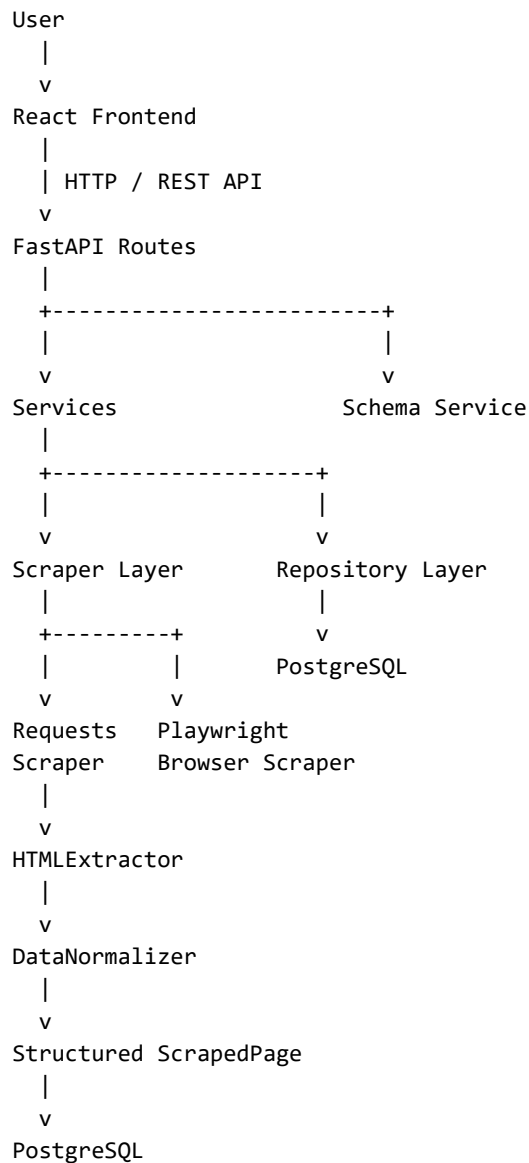
4. Objectives

- Build a reusable website scraping service that accepts user-provided URLs.
- Support ordinary HTML pages and many JavaScript-rendered pages.
- Convert scraped content into a consistent structured data model.

- Persist scraped records in PostgreSQL using parameterized SQL.
- Provide REST APIs for Create, Read, Update and Delete operations.
- Provide APIs to inspect available database tables and their columns/data types.
- Provide CSV export through a separate Python script and an API endpoint.
- Use clean separation of responsibilities so individual components can be maintained or extended.
- Provide a React interface for demonstrating the complete workflow.

5. System Architecture

The project follows a layered architecture. Each layer has a focused responsibility. The API layer receives HTTP requests, services contain business logic, scrapers retrieve and parse website content, repositories communicate with PostgreSQL, and the frontend consumes the APIs.



5.1 Main Layers

Layer	Responsibility
Frontend	User interface, forms, navigation, tables and API calls.

API / Routes	Defines HTTP endpoints, validates input and returns responses.
Services	Contains application/business logic and coordinates components.
Scrapers	Retrieves webpage content using HTTP or a real browser.
Extractor / Normalizer	Converts HTML/raw dictionaries into a consistent structure.
Repository	Performs database CRUD operations and isolates SQL/database details.
Database	Persists scraped records in PostgreSQL.
Core	Configuration, logging and custom exceptions.

6. Project Structure

```

universal-web-scraper/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── scrape_routes.py
│   │   ├── crud_routes.py
│   │   ├── schema_routes.py
│   │   └── csv_routes.py
│   ├── core/
│   │   ├── config.py
│   │   ├── exceptions.py
│   │   └── logger.py
│   ├── models/
│   │   └── scraped_data.py
│   ├── schemas/
│   │   ├── scrape_schema.py
│   │   └── response_schema.py
│   ├── services/
│   │   ├── scraper_service.py
│   │   ├── database_service.py
│   │   ├── csv_service.py
│   │   └── schema_service.py
│   ├── scrapers/
│   │   ├── base_scraper.py
│   │   ├── requests_scraper.py
│   │   ├── browser_scraper.py
│   │   ├── html_extractor.py
│   │   └── scraper_factory.py
│   ├── repositories/
│   │   ├── base_repository.py
│   │   └── scraped_data_repository.py
│   └── database/
│       ├── connection.py
│       └── migrations.py
├── scripts/
│   └── generate_csv.py
├── frontend/
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md

```

7. Backend Working

7.1 Application Entry Point - main.py

main.py creates the FastAPI application. It registers the routers, configures CORS, exposes basic root/health endpoints and initializes the database table during application startup.

- Creates the FastAPI application instance.
- Registers scrape, CRUD, schema and CSV routers.
- Enables CORS so the React frontend can communicate with the backend.
- Runs table creation during application startup.
- Provides health/root endpoints for basic verification.

7.2 Configuration

core/config.py uses Pydantic Settings to load configuration from environment variables. Database credentials are therefore kept outside the source code.

```
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_NAME=web_scraper
DATABASE_USER=postgres
DATABASE_PASSWORD=*****
REQUEST_TIMEOUT=15
```

The .env file is excluded from Git using .gitignore, while .env.example documents the required variables.

7.3 Scraper Abstraction

BaseScraper defines the common contract for scraper implementations. It is an abstract class with a scrape(url) method. This allows different scraping strategies to be used without changing the service layer.

```
class BaseScraper(ABC):
    @abstractmethod
    def scrape(self, url: str) -> list[dict[str, Any]]:
        pass
```

7.4 RequestsScraper

RequestsScraper handles normal HTTP pages. It sends an HTTP request, validates the response, and passes the returned HTML to the HTML extractor. It is lightweight and faster than launching a browser.

- Uses requests for HTTP communication.
- Uses BeautifulSoup for HTML parsing.
- Handles timeout/network/request failures.
- Checks HTTP status using response.raise_for_status().
- Extracts title, headings, paragraphs and links.

7.5 BrowserScraper - Dynamic Website Support

BrowserScraper uses Playwright with Chromium. Unlike a simple HTTP client, Playwright loads the page in a browser environment and executes JavaScript. This allows the application to work with many websites whose visible content is produced after page load.

Open URL
↓

```
Chromium loads page
↓
JavaScript executes
↓
Rendered HTML becomes available
↓
BeautifulSoup parses rendered HTML
↓
Structured content is extracted
```

This does not mean that the project captures every network request or every Fetch/XHR/GraphQL endpoint. The current implementation extracts the resulting page content. Network interception would be an optional future enhancement.

7.6 GenericScraper / Fallback Strategy

The GenericScraper coordinates the HTTP and browser strategies. It first attempts the lightweight RequestsScraper. If the request fails or the extracted content is insufficient for the target page, it can fall back to BrowserScraper.

```
URL
↓
RequestsScraper
↓
Content sufficient?
├─ Yes → Return data
└─ No  → BrowserScraper
      ↓
    Return data
```

7.7 HTMLExtractor

HTMLExtractor centralizes HTML parsing logic so RequestsScraper and BrowserScraper do not duplicate the same BeautifulSoup extraction code. It extracts the page title, headings, paragraphs and links.

7.8 DataNormalizer

The DataNormalizer converts raw scraper dictionaries into the application's Pydantic data model. This creates a predictable structure before data reaches the database.

7.9 Pydantic Schemas

```
ScrapedPage
├─ url
├─ title
├─ headings[]
├─ paragraphs[]
├─ links[]
└─ scraped_at

ScrapedLink
├─ text
└─ href
```

Pydantic validation ensures that the API and internal data flow use a defined structure. The URL is validated as a URL rather than being accepted as an arbitrary string.

8. PostgreSQL Database

The application uses PostgreSQL through `psycopg2`. A `scraped_pages` table stores the normalized page data.

```
CREATE TABLE IF NOT EXISTS scraped_pages (  
    id SERIAL PRIMARY KEY,  
    url TEXT NOT NULL,  
    title TEXT,  
    headings JSONB DEFAULT '[]'::jsonb,  
    paragraphs JSONB DEFAULT '[]'::jsonb,  
    links JSONB DEFAULT '[]'::jsonb,  
    scraped_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

JSONB is used for headings, paragraphs and links because these fields naturally contain variable-length lists/structured values. `psycopg2.extras.Json` is used when sending Python list/dictionary values to PostgreSQL.

8.1 Database Connection

`database/connection.py` provides a connection context manager. It reads database settings from configuration, opens a PostgreSQL connection, handles database errors and closes the connection safely.

8.2 Repository Pattern

The repository layer isolates SQL queries from the service and route layers. `BaseRepository` defines the CRUD contract, while `ScrapedDataRepository` implements the PostgreSQL-specific operations.

```
API Route  
↓  
Service  
↓  
Repository  
↓  
Parameterized SQL  
↓  
PostgreSQL
```

Parameterized SQL is used instead of building SQL strings by concatenating user input. This improves query safety and keeps database operations clean.

9. API Documentation

9.1 Scraping API

Method	Endpoint	Purpose
POST	/scrape/	Accept URL, scrape website and save extracted records.

Request:

```
{
  "url": "https://example.com"
}
```

Typical response:

```
{
  "scraped_count": 1,
  "records": [...]
}
```

9.2 CRUD APIs

Method	Endpoint	Purpose
POST	/data/	Create a new scraped-data record.
GET	/data/	Return all stored records.
GET	/data/{record_id}	Return one record by ID.
PUT	/data/{record_id}	Update an existing record.
DELETE	/data/{record_id}	Delete an existing record.

9.3 Schema APIs

Method	Endpoint	Purpose
GET	/schema/tables	List tables in the public PostgreSQL schema.
GET	/schema/tables/{table_name}	Return columns, data types, nullability and defaults.

9.4 CSV API

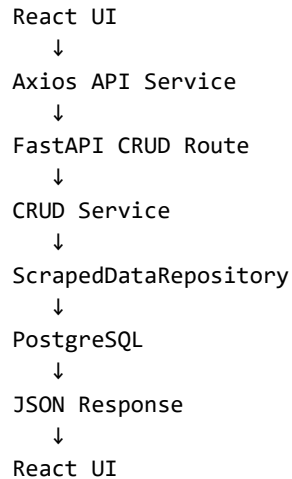
Method	Endpoint	Purpose
GET	/export/csv	Generate and download scraped data as CSV.

10. End-to-End Scraping Flow

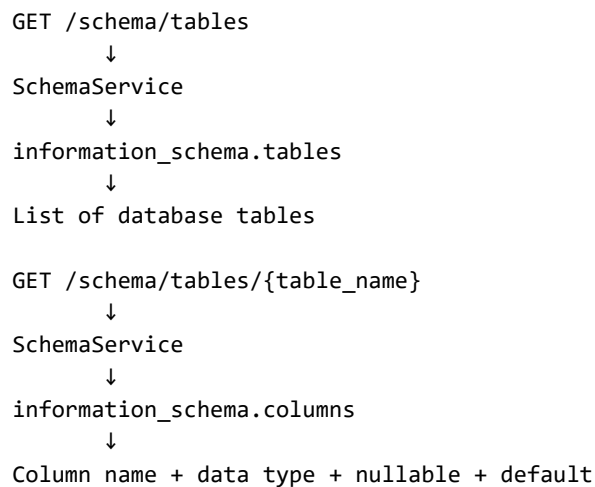
1. User enters website URL in React frontend.
2. Frontend sends POST /scrape/.
3. FastAPI validates the URL using ScrapeRequest.
4. Scrape route creates/injects the scraper and repository dependencies.
5. ScrapeService calls the scraper.
6. RequestsScraper tries normal HTTP scraping.
7. If required, BrowserScraper loads the page using Playwright.
8. HTMLExtractor extracts title, headings, paragraphs and links.
9. DataNormalizer converts raw data into ScrapedPage objects.
10. ScrapeService sends normalized data to ScrapedDataRepository.

11. Repository executes parameterized INSERT statements.
12. PostgreSQL stores the record.
13. API returns the saved records.
14. React displays the result to the user.

11. CRUD Flow

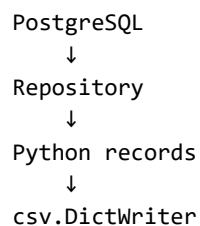


12. Schema Flow



13. CSV Generation Flow

The assignment requires a separate Python script and the csv module. The project implements scripts/generate_csv.py for this purpose. It obtains records from the repository, creates a CSV file with headers and serializes structured fields such as headings, paragraphs and links into readable values.



↓
scraped_data.csv

14. Frontend Architecture

The frontend is implemented using React and Vite with Tailwind CSS. Axios is used for HTTP requests and React Router handles navigation.

Page / Component	Purpose
Dashboard	Shows record count, website count, latest data and system status.
ScrapeWebsite	Accepts a URL and starts the scraping operation.
ScrapedData	Lists saved records, supports search, refresh and delete.
RecordDetails	Displays one record and supports edit/update mode.
DatabaseSchema	Displays database tables and selected table columns.
DataTable	Reusable tabular display component.
ExportCsvButton	Downloads the CSV from the backend.
Navbar / Sidebar	Application navigation.
Loading / ErrorMessage	Reusable loading and error UI.
services/api.js	Centralized Axios/API communication layer.

15. Frontend-to-Backend Communication

React Component
↓
services/api.js
↓
Axios
↓
http://127.0.0.1:8000
↓
FastAPI
↓
PostgreSQL / Scraper
↓
JSON / File Response
↓
React Component

CORS is configured in FastAPI to allow the development frontend running at localhost:5173 to communicate with the backend running at localhost:8000.

16. Design Principles and Patterns

16.1 SOLID Principles

Principle	How the Project Applies It
S - Single Responsibility	Routes, services, scrapers, repositories, configuration and extraction have separate responsibilities.
O - Open/Closed	New scraper implementations can be added through the BaseScraper abstraction without rewriting service logic.
L - Liskov Substitution	RequestsScraper and BrowserScraper implement the BaseScraper contract.
I - Interface Segregation	Small focused abstractions such as BaseRepository and

	BaseScraper are used.
D - Dependency Inversion	Services depend on abstractions such as BaseScraper/BaseRepository and receive dependencies through construction.

16.2 Design Patterns

Pattern	Usage
Factory Pattern	ScraperFactory creates the scraper strategy used by the application.
Repository Pattern	ScrapedDataRepository isolates SQL and database operations.
Dependency Injection	Routes/services receive scraper and repository dependencies rather than hard-coding them.
Strategy-like Scraper Design	Different scraper implementations can perform the same scrape operation through a common interface.

17. Error Handling and Logging

The backend defines custom exceptions such as ScraperException, InvalidURLException, NetworkException, ParsingException, DatabaseException, DatabaseConnectionException and DatabaseQueryException. These exceptions make failures easier to identify and handle at the appropriate layer.

- Invalid URL → validation/custom exception.
- Network timeout/request failure → network/scraper exception.
- Parsing problem → parsing exception.
- Database connection problem → database connection exception.
- SQL/query problem → database query exception.
- Application events and failures → centralized project logger.

18. Security and Reliability Considerations

- Database credentials are stored in environment variables rather than committed source code.
- The .env file is excluded through .gitignore.
- SQL queries use parameters rather than string concatenation.
- Pydantic validates incoming URL/request structures.
- HTTP request timeout is configurable.
- HTTP status failures are checked before processing responses.
- The scraper is designed for publicly accessible pages; authentication, CAPTCHA, anti-bot systems and unusual site behavior can prevent successful scraping.

19. Current Scope and Limitations

The word 'universal' in this project means that the application accepts arbitrary user-provided public URLs rather than being hard-coded to one domain. It does not guarantee successful scraping of every website.

- Current BrowserScraper extracts rendered page content; it does not separately record every Fetch/XHR/GraphQL network endpoint.

- Sites protected by CAPTCHA or strong anti-bot controls may not be scrapeable.
- Authenticated/private content requires appropriate access and is not automatically bypassed.
- Some content loaded only after user interaction may require additional Playwright actions.
- WebSockets and non-HTML application protocols are outside the current extraction model.

20. Possible Future Enhancements

- Capture Fetch/XHR requests and responses through Playwright network listeners.
- Detect and store GraphQL endpoints and request metadata.
- Support pagination and infinite scrolling.
- Allow user-defined CSS selectors or extraction rules.
- Add scheduled scraping jobs.
- Add authentication and role-based access if required.
- Add rate limiting and robots.txt-aware controls.
- Add richer data export formats such as JSON and Excel.
- Add Docker deployment and production configuration.

21. Installation and Execution

21.1 Backend Setup

```
python -m venv venv
# Windows
venv\Scripts\activate

pip install -r requirements.txt

python -m playwright install chromium

uvicorn app.main:app --reload
```

21.2 PostgreSQL Setup

1. Install and start PostgreSQL.
2. Create the web_scraper database.
3. Set DATABASE_HOST, DATABASE_PORT, DATABASE_NAME, DATABASE_USER and DATABASE_PASSWORD in .env.
4. Start the FastAPI application.
5. The application creates the scraped_pages table during startup.

21.3 Frontend Setup

```
cd frontend
npm install
npm run dev
```

21.4 API Documentation

When FastAPI is running, the interactive Swagger UI is available at the backend's /docs route. It can be used during the Loom demonstration to show and test the APIs.

22. Conclusion

The Universal Web Scraper combines web scraping, browser automation, REST API development, PostgreSQL persistence, CSV generation and a React interface in a single maintainable application. The architecture separates responsibilities across API, service, scraper, repository and database layers, which makes the project easier to explain, maintain and extend. The implementation satisfies the core assignment workflow while leaving clear extension points for advanced capabilities such as network request interception and GraphQL endpoint discovery.